

Chapter 5: I/O Systems

How fast is I/O hardware?

The data rate mentioned below don't reflect the technological advancement at the present time. They are presented just for the sake of demonstrating how I/O activities are slow.

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
Printer / scanner	200 KB/sec
USB	1.5 MB/sec
Digital camcorder	4 MB/sec
Fast Ethernet	100 MB/sec
Hard drive	20 MB/sec
FireWire (IEEE 1394)	50 MB/sec
XGA monitor	60 MB/sec
PCI bus	500 MB/sec

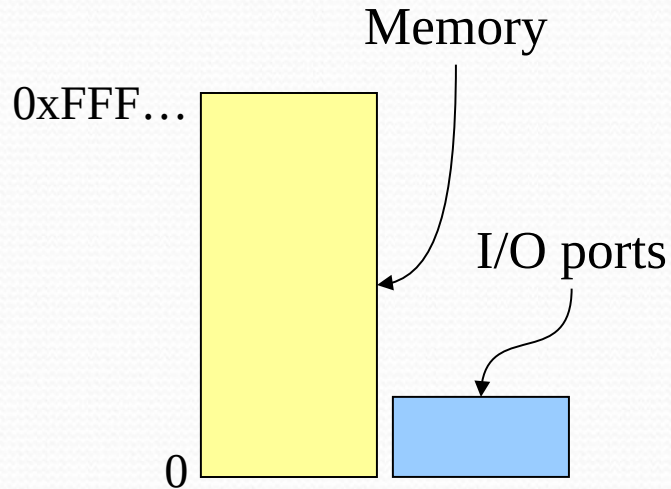
Characteristics of I/O Devices

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read–write	CD-ROM graphics controller disk

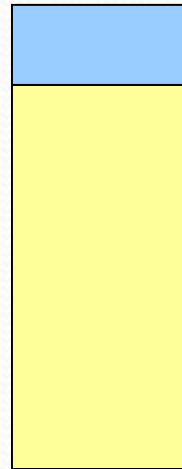
Device controllers

- I/O devices have components
 - Mechanical component
 - Electronic component
- Electronic component controls the device
 - May be able to handle multiple devices
 - May be more than one controller per mechanical component (example: hard drive)
- Controller's tasks
 - Convert serial bit stream to block of bytes
 - Perform error correction as necessary
 - Make available to main memory

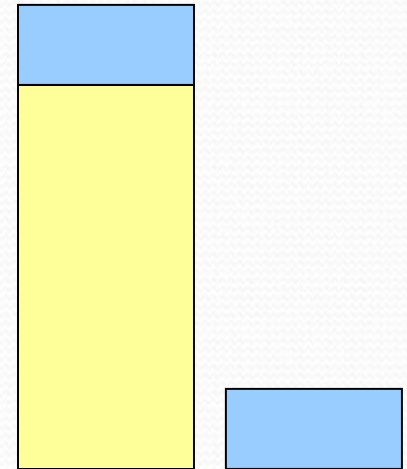
Memory-Mapped I/O



Separate
I/O & memory
space



Memory-mapped I/O

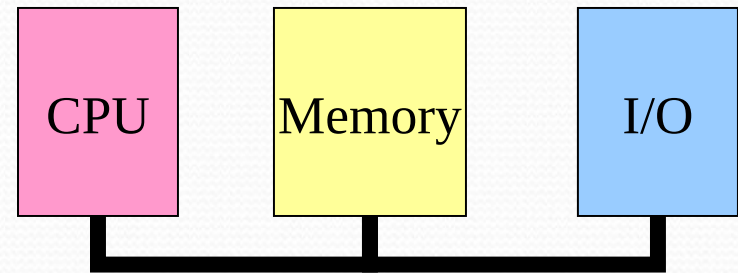


Hybrid: both
memory-mapped &
separate spaces

How is memory-mapped I/O done?

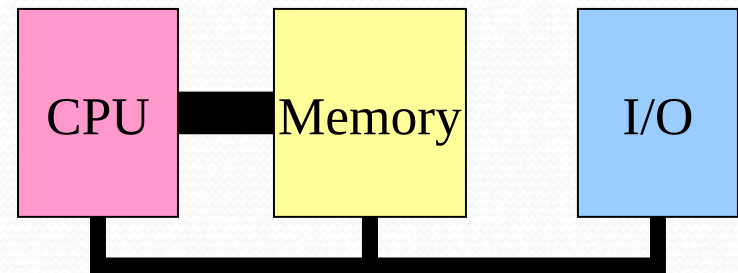
- Single-bus

- All memory accesses go over a shared bus
- I/O and RAM accesses compete for bandwidth



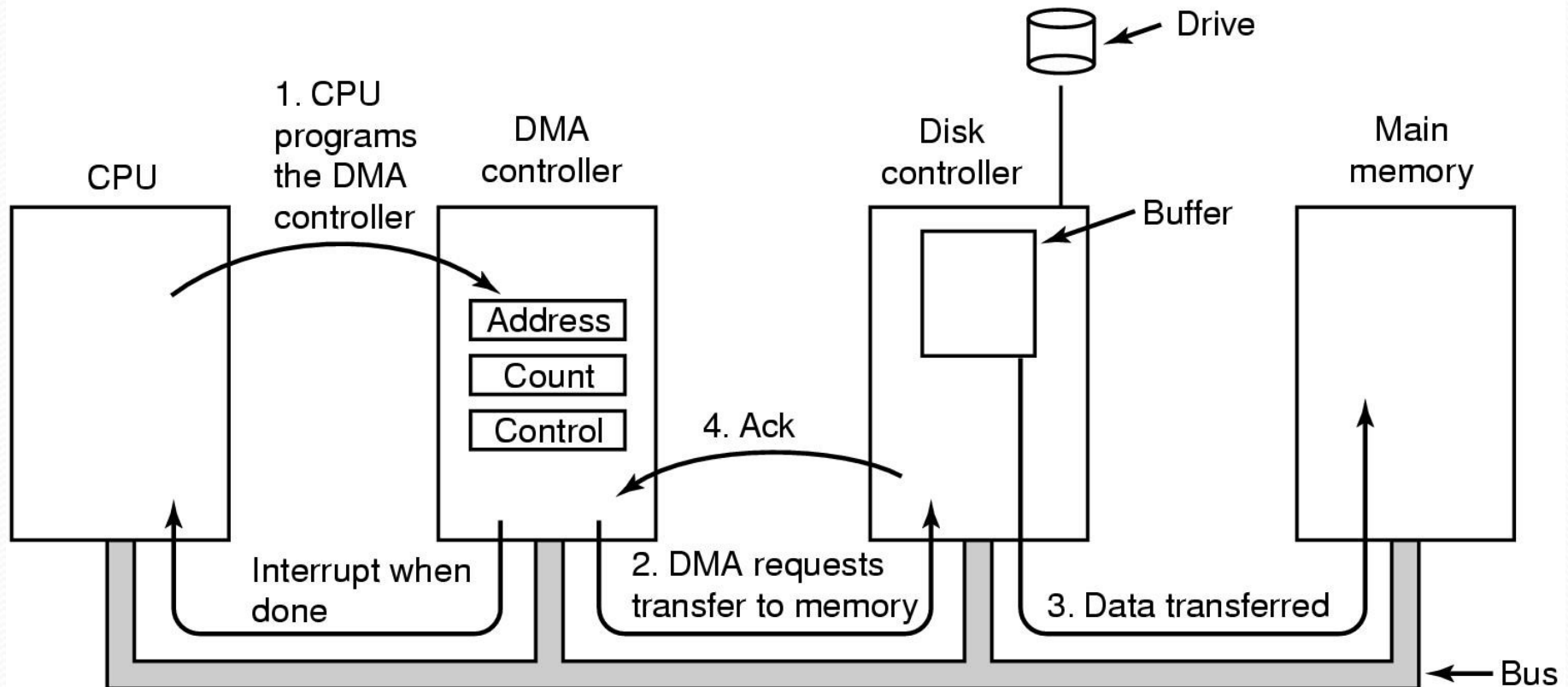
- Dual-bus

- RAM access over high-speed bus
- I/O access over lower-speed bus
- Less competition
- More hardware (more expensive...)



↖
This port allows I/O devices access into memory

Direct Memory Access (DMA) operation



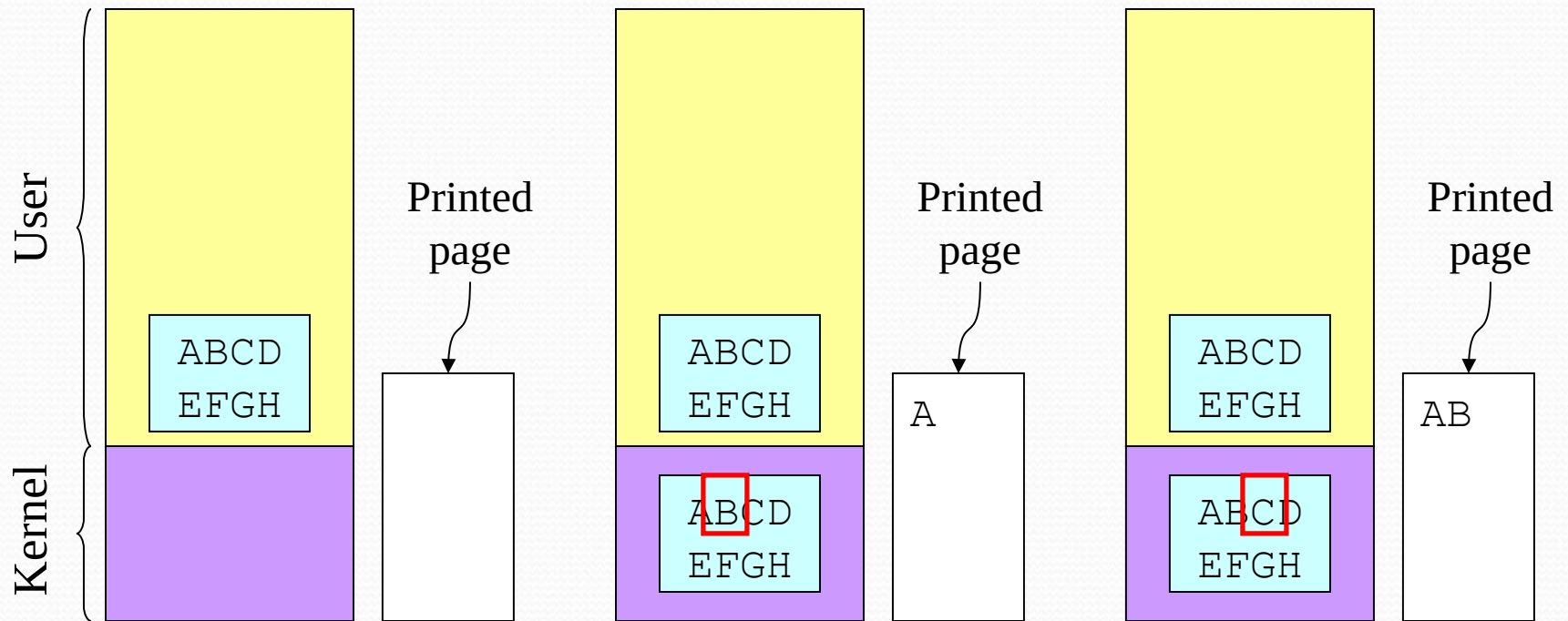
I/O software: goals

- Device independence
 - Programs can access any I/O device
 - No need to specify device in advance
- Uniform naming
 - Name of a file or device is a string or an integer
 - Doesn't depend on the machine (underlying hardware)
- Error handling
 - Done as close to the hardware as possible
 - Isolate higher-level software
- Synchronous vs. asynchronous transfers
 - Blocked transfers vs. interrupt-driven
- Buffering
 - Data coming off a device cannot be stored in final destination
- Sharable vs. dedicated devices

Programmed I/O

- There are three fundamentally different ways that I/O can be performed
 - Programmed I/O,
 - interrupt-driven I/O and
 - I/O using DMA

Programmed I/O: printing a page



Code for programmed I/O

```
copy_from_user (buffer, p, count); // copy into kernel buffer
for (j = 0; j < count; j++) { // loop for each char
    while (*printer_status_reg != READY)
        ; // wait for printer to be ready
    *printer_data_reg = p[j]; // output a single character
}
return_to_user();
```

Interrupt-Driven I/O

- Consider the case of printing on a printer that does not buffer characters but prints each one as it arrives:
 - The CPU will be left idle for a significant amount of time for waiting to receive the next character
 - Solution: use of interrupt so that CPU can work on other things until the characters are buffered for printing.

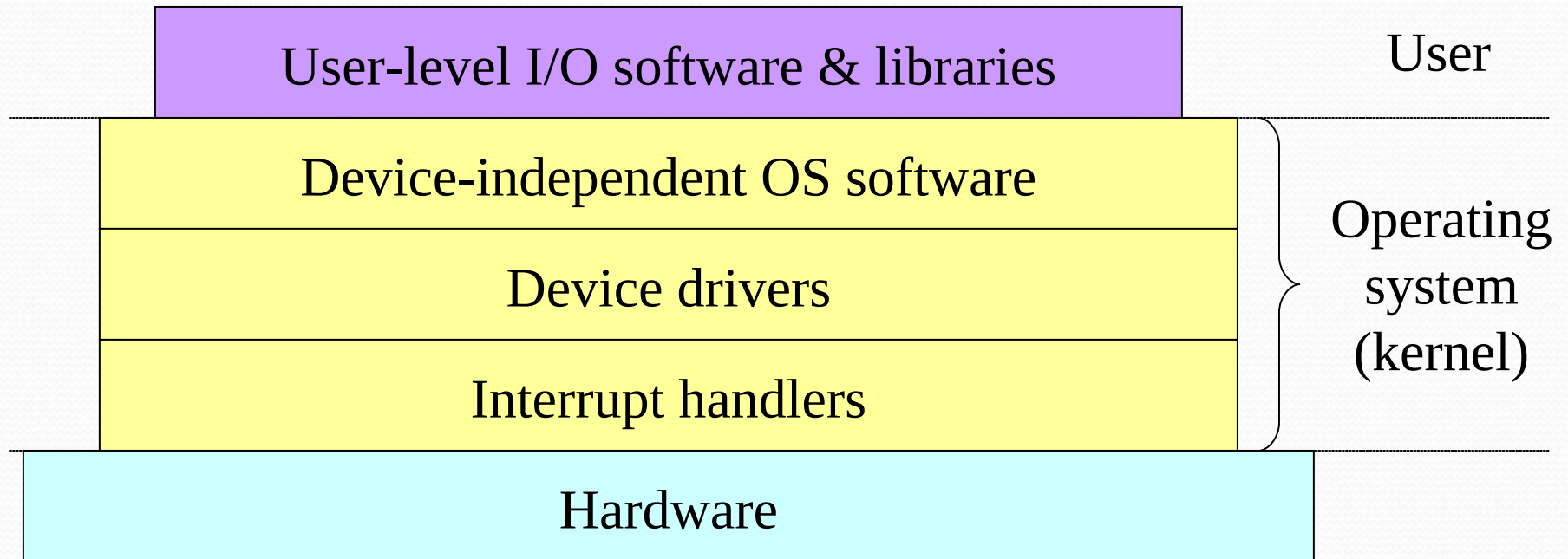
Using DMA

- The problem with interrupt driven I/O is that an interrupt occurs on every character.
- The solution is to use DMA:
 - Let the DMA controller feed the characters to the printer one at time, without the CPU being bothered.

Assignment

- Programmed I/O
- Interrupt Driven I/O
- I/O using DMA
- **Note: Present a comparative analysis of the three I/O techniques**

Layers of I/O software

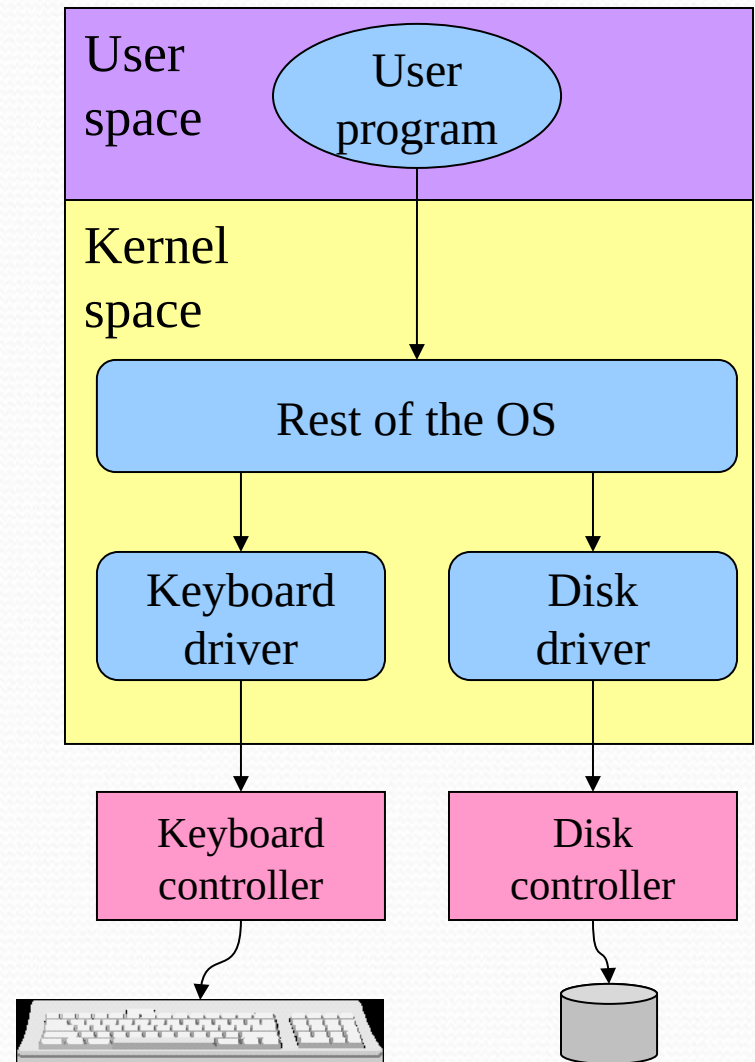


Interrupt handlers

- Interrupt handlers are best hidden
 - Driver starts an I/O operation and blocks
 - Interrupt notifies of completion
- Interrupt procedure does its task
 - Then unblocks driver that started it
 - Perform minimal actions at interrupt time
 - Some of the functionality can be done by the driver after it is unblocked
- Interrupt handler must
 - Save regs not already saved by interrupt hardware
 - Set up context for interrupt service procedure

Device drivers

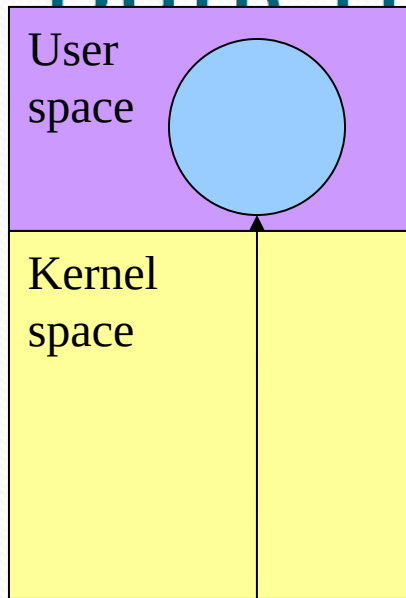
- Device drivers go between device controllers and rest of OS
 - Drivers standardize interface to widely varied devices
- Device drivers communicate with controllers over bus
 - Controllers communicate with devices themselves



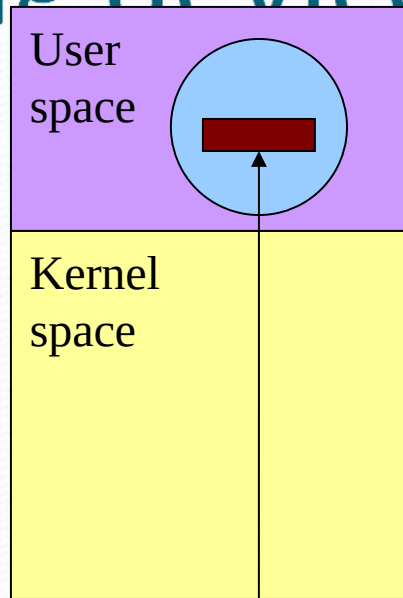
Device-independent I/O software

- Device-independent I/O software provides common “library” routines for I/O software
- Helps drivers maintain a standard appearance to the rest of the OS
- Uniform interface for many device drivers for
 - Buffering
 - Error reporting
 - Allocating and releasing dedicated devices
 - Suspending and resuming processes
- Common resource pool
 - Device-independent block size (keep track of blocks)
 - Other device driver resources

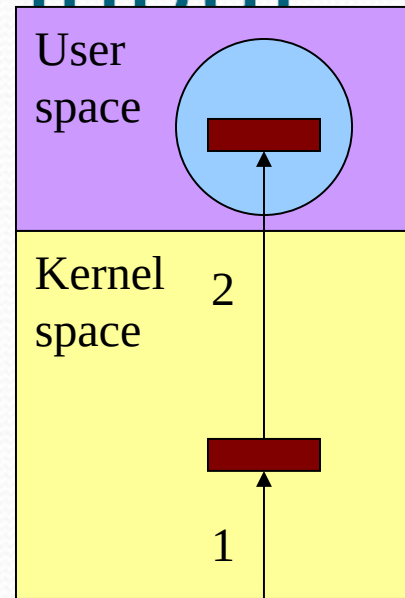
Buffering device input



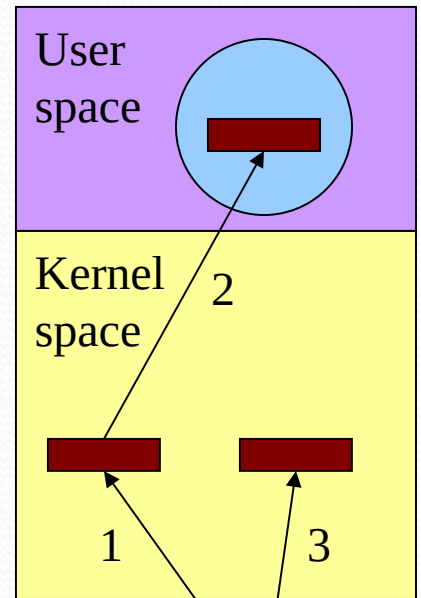
Unbuffered
input



Buffering in
user space

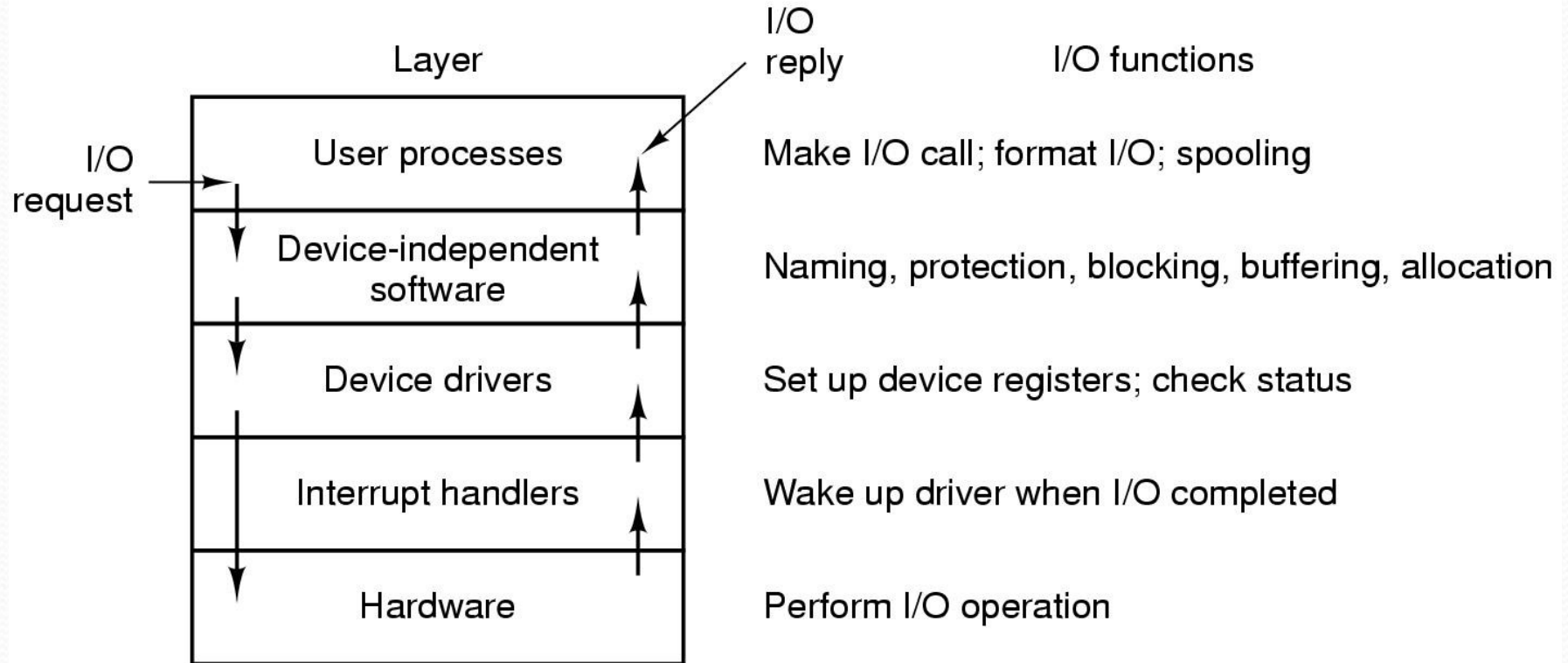


Buffer in kernel
Copy to user space



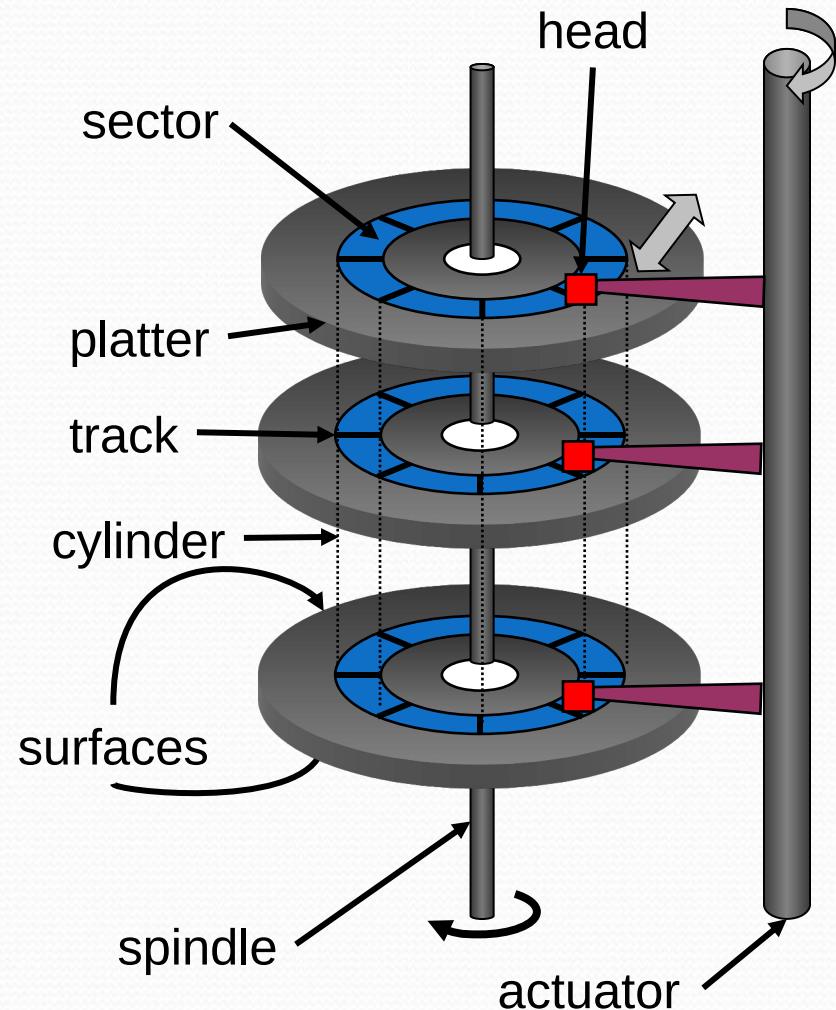
Double buffer
in kernel

Anatomy of an I/O request



Disk drive structure

- Data stored on surfaces
 - Up to two surfaces per platter
 - One or more platters per disk
- Data in concentric tracks
 - Tracks broken into sectors
 - 256B-1KB per sector
 - Cylinder: corresponding tracks on all surfaces
- Data read and written by heads
 - Actuator moves heads
 - Heads move in unison

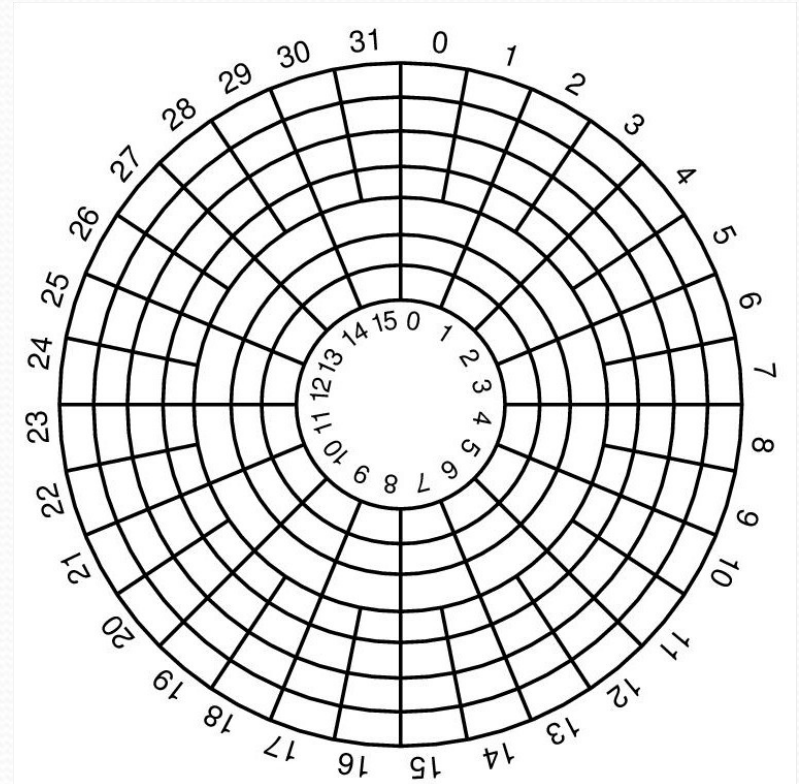


Disk drive specifics

	IBM 360KB floppy	WD 18GB HD
Cylinders	40	10601
Tracks per cylinder	2	12
Sectors per track	9	281 (average)
Sectors per disk	720	35742000
Bytes per sector	512	512
Capacity	360 KB	18.3 GB
Seek time (minimum)	6 ms	0.8 ms
Seek time (average)	77 ms	6.9 ms
Rotation time	200 ms	8.33 ms
Spinup time	250 ms	20 sec
Sector transfer time	22 ms	17 μ sec

Disk “zones”

- Outside tracks are longer than inside tracks
- Two options
 - Bits are “bigger”
 - More bits (transfer faster)
- Modern hard drives use second option
 - More data on outer tracks
- Disk divided into “zones”
 - Constant sectors per track in each zone



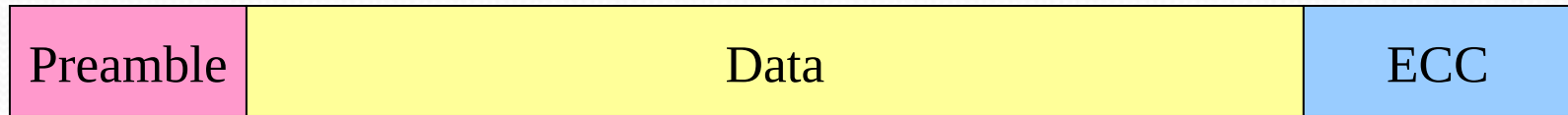
Disk “addressing”

- Millions of sectors on the disk must be labeled
- Two possibilities
 - Cylinder/track/sector
 - Sequential numbering
- Modern drives use sequential numbers
 - Disks map sequential numbers into specific location
 - Mapping may be modified by the disk
 - Remap bad sectors
 - Optimize performance
 - Hide the exact geometry, making life simpler for the OS

Building a better “disk”

- Problem: CPU performance has been increasing exponentially, but disk performance hasn't
 - Disks are limited by mechanics
- Problem: disks aren't all that reliable
- Solution: distribute data across disks, and use some of the space to improve reliability
 - Data transferred in parallel
 - Data stored across drives (*striping*)
 - Parity on an “extra” drive for reliability

Structure of a disk sector



- Preamble contains information about the sector
 - Sector number & location information
- Data is usually 256, 512, or 1024 bytes
- ECC (Error Correcting Code) is used to detect & correct minor errors in the data

Assignment

- Improving the performance of a hardisk
 - Conventional techniques
 - RAID- 0,1,5
- Security issues in operating systems
 - Security problems
 - Security mechanisms

Assignment

- Disk “Zones”
 - Zone bit recording